Question 2: Java program for matrix multiplication using Hadoop MapReduce

MapReduce

Aim

To multiply two matrices using Hadoop MapReduce and display the resultant matrix.

Procedure

1. Create two input files, matrixA.txt and matrixB.txt. Each record contains matrix name, row index, column index and

value.

2. Compile the Java program with the Hadoop classpath and package the classes into a JAR file.

3. Copy the input directory to HDFS and execute the MapReduce job.

4. The mapper emits each A(i,k) value for every output column and each B(k,j) value for every output row.

5. The reducer groups values by output cell (i,j), multiplies matching A(i,k) and B(k,j) values, and sums the products.

6. Read the output directory from HDFS.

Input format

A,0,0,1

A,0,1,2

A,1,0,3

A,1,1,4

B,0,0,5

B,0,1,6

B,1,0,7

B,1,1,8

Program

```
import java.io.IOException;

import java.util.*;

import org.apache.hadoop.conf.Configuration;

import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.*;

import org.apache.hadoop.mapreduce.*;
```

```
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
public class MatrixMultiply {
public static class M extends Mapper<LongWritable,Text,Text,Text> {
public void map(LongWritable key, Text value, Context c)
throws IOException, InterruptedException {
String[] x = value.toString().trim().split(",");
String matrix = x[0];
int r = Integer.parseInt(x[1]);
int col = Integer.parseInt(x[2]);
int v = Integer.parseInt(x[3]);
int n = c.getConfiguration().getInt("matrix.size", 2);
if (matrix.equals("A")) {
for (int j=0; j<n; j++)
c.write(new Text(r+","+j), new Text("A,"+col+","+v));
} else {
for (int i=0; i<n; i++)
c.write(new Text(i+","+col), new Text("B,"+r+","+v));
}
}
}
public static class R extends Reducer<Text,Text,Text,IntWritable> {
public void reduce(Text key, Iterable<Text> values, Context c)
throws IOException, InterruptedException {
Map<Integer,Integer> a = new HashMap<>();
Map<Integer,Integer> b = new HashMap<>();
for (Text t : values) {
String[] x = t.toString().split(",");
int k = Integer.parseInt(x[1]);
int v = Integer.parseInt(x[2]);
if (x[0].equals("A")) a.put(k,v);
```

```
else b.put(k,v);
}
int sum = 0;
for (Integer k : a.keySet())
if (b.containsKey(k)) sum += a.get(k) * b.get(k);
c.write(key, new IntWritable(sum));
}
}
public static void main(String[] args) throws Exception {
Configuration conf = new Configuration();
conf.setInt("matrix.size", 2); // Change for other square matrix sizes
Job job = Job.getInstance(conf, "Matrix Multiplication");
job.setJarByClass(MatrixMultiply.class);
job.setMapperClass(M.class);
job.setReducerClass(R.class);
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(Text.class);
job.setMapOutputKeyClass(Text.class);
job.setMapOutputValueClass(Text.class);
FileInputFormat.addInputPath(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));
System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}
```

Compile and execute

```
export HADOOP_CLASSPATH=$(hadoop classpath)
javac -classpath "$HADOOP_CLASSPATH" -d classes MatrixMultiply.java
jar -cvf matrixmultiply.jar -C classes .
hdfs dfs -mkdir -p /input/matrix
hdfs dfs -put matrixA.txt /input/matrix/
hdfs dfs -put matrixB.txt /input/matrix/
```

hadoop jar matrixmultiply.jar MatrixMultiply /input/matrix /output/matrix

hdfs dfs -cat /output/matrix/part-r-00000

Output of Program

0,0 19

0,1 22

1,0 43

1,1 50

Result

The MapReduce program successfully multiplies the two 2×2 matrices and produces the resultant matrix [[19, 22], [43,

50]].